

🏗️ Introducción al Service Layer

🎯 Objetivos de Aprendizaje

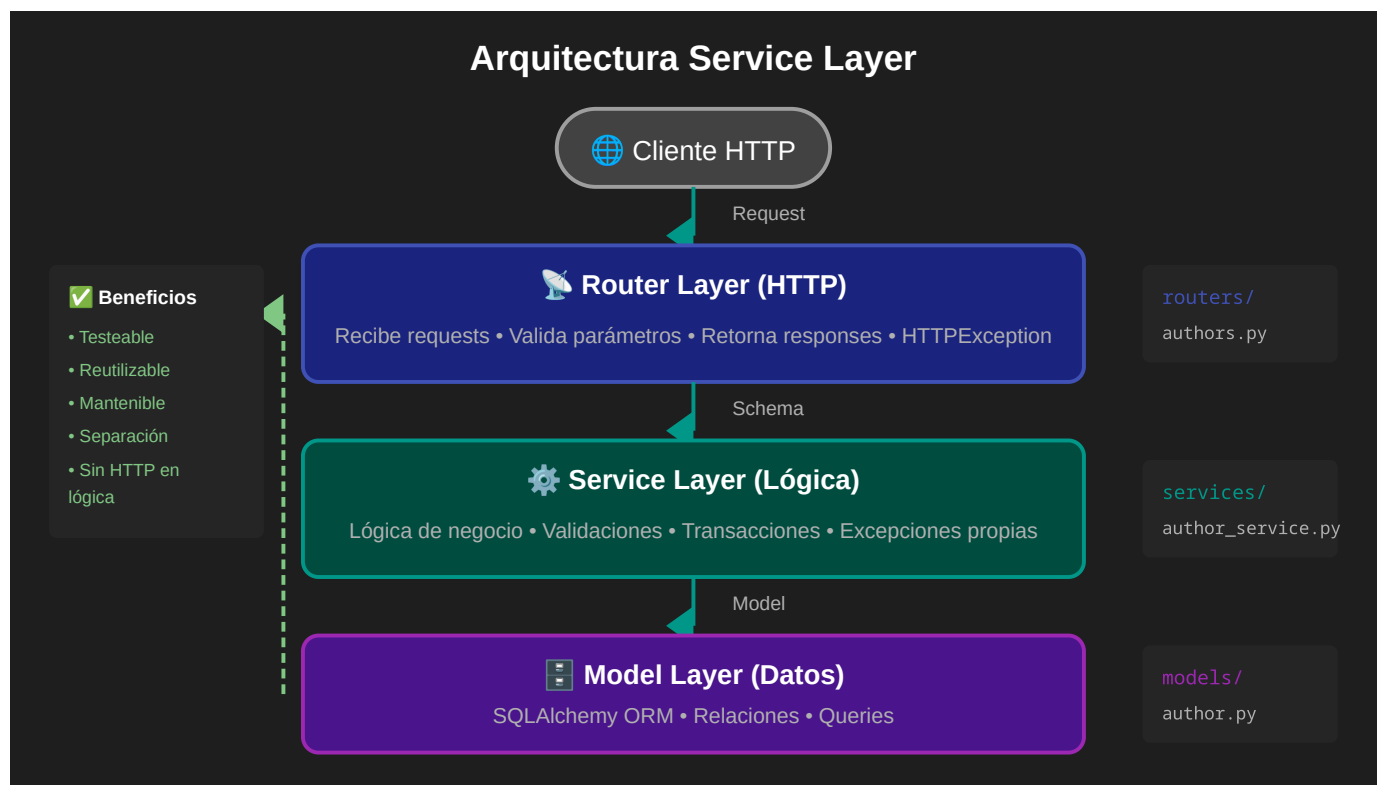
Al finalizar este tema, serás capaz de:

- ✅ Entender qué es el Service Layer y por qué usarlo
- ✅ Identificar problemas de código sin separación de capas
- ✅ Conocer la arquitectura de capas en FastAPI
- ✅ Diferenciar responsabilidades de cada capa

📖 Contenido

1. ¿Qué es el Service Layer?

El **Service Layer** es un patrón arquitectónico que **separa la lógica de negocio** de los endpoints HTTP.



2. El Problema: Endpoints "Fat"

Sin Service Layer, todo está en el endpoint:

```
# ❌ MAL: Endpoint "gordo" con todo mezclado
@app.post("/posts", response_model=PostResponse)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    # Validación de negocio
    if len(post.title) < 5:
        raise HTTPException(400, "Title too short")

    # Verificar que el autor existe
    author = db.get(Author, post.author_id)
```

```

if not author:
    raise HTTPException(404, "Author not found")

# Verificar que el autor puede publicar
if not author.is_active:
    raise HTTPException(403, "Author is not active")

# Procesar tags
tags = []
for tag_name in post.tag_names:
    tag = db.execute(
        select(Tag).where(Tag.name == tag_name)
    ).scalar_one_or_none()
    if not tag:
        tag = Tag(name=tag_name)
        db.add(tag)
    tags.append(tag)

# Crear el post
db_post = Post(
    title=post.title,
    content=post.content,
    author_id=post.author_id,
    tags=tags
)
db.add(db_post)
db.commit()
db.refresh(db_post)

# Enviar notificación (más lógica de negocio)
send_notification(author.email, f"Post '{post.title}' created")

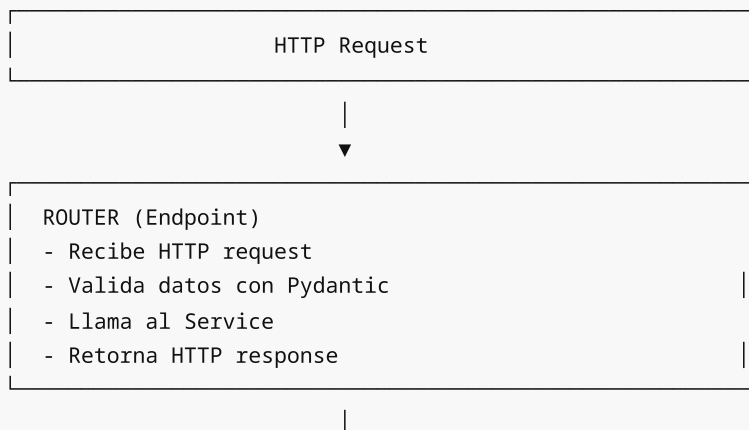
return db_post

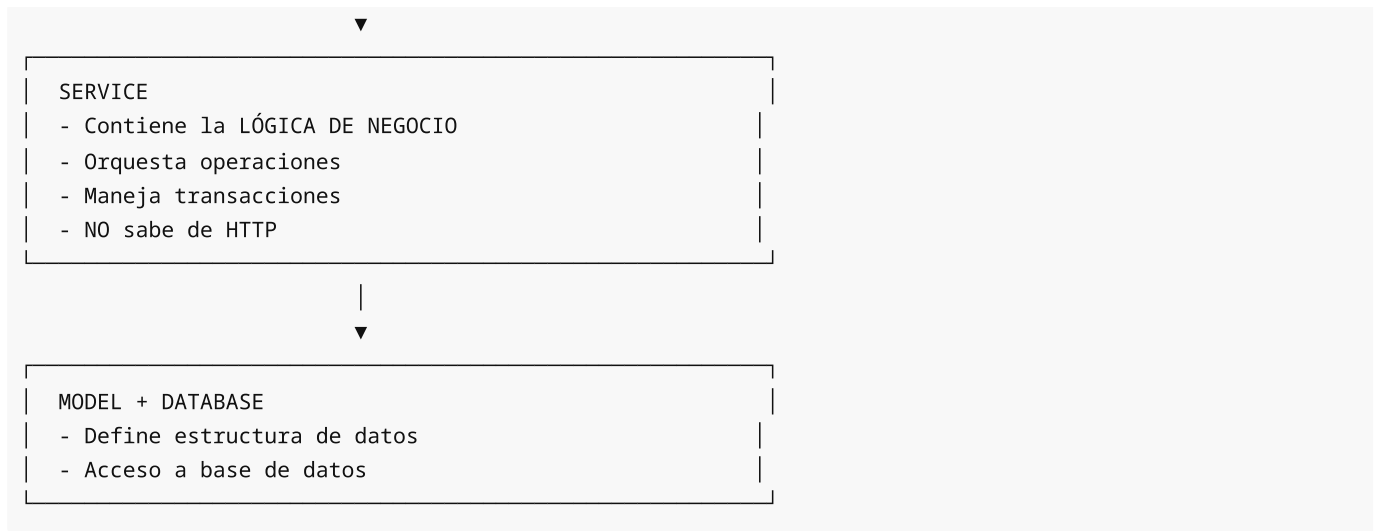
```

Problemas:

- 🔥 Difícil de testear (necesitas HTTP para probar lógica)
- 🔥 Difícil de reutilizar (no puedes crear posts desde otro lugar)
- 🔥 Difícil de mantener (50+ líneas de código mezclado)
- 🔥 Difícil de entender (HTTP + negocio + DB mezclados)

3. La Solución: Separación de Capas





4. Beneficios del Service Layer

Beneficio	Descripción
Testabilidad	Puedes testear lógica sin HTTP
Reutilización	La misma lógica para API, CLI, workers
Mantenibilidad	Código organizado por responsabilidad
Claridad	Fácil entender qué hace cada parte
Escalabilidad	Fácil agregar nuevas features

5. Estructura de Carpetas

```
src/
├─ main.py           # FastAPI app, configuración
├─ database.py       # Engine, Session, Base
├─
├─ routers/         # 🌐 Capa HTTP
│  └─ __init__.py
│  └─ authors.py    # Endpoints de autores
│  └─ posts.py      # Endpoints de posts
├─
├─ services/        # 📁 Capa de Negocio
│  └─ __init__.py
│  └─ author_service.py
│  └─ post_service.py
├─
├─ models/          # 📦 Capa de Datos
│  └─ __init__.py
│  └─ author.py
│  └─ post.py
├─
└─ schemas/         # 📋 DTOs (validación)
   └─ __init__.py
   └─ author.py
   └─ post.py
```

6. Responsabilidades de Cada Capa

Routers (Endpoints)

```
# routers/posts.py
# ✅ SOLO maneja HTTP
@router.post("/", response_model=PostResponse, status_code=201)
def create_post(
    post: PostCreate,
    db: Session = Depends(get_db)
):
    """Endpoint HTTP - delega al service"""
    service = PostService(db)
    return service.create(post)
```

Responsabilidades:

- Definir rutas HTTP
- Validar entrada con Pydantic
- Llamar al Service
- Manejar errores HTTP
- Retornar respuestas

NO debe:

- Contener lógica de negocio
- Hacer queries directamente
- Conocer detalles de implementación

Services

```
# services/post_service.py
# ✅ SOLO lógica de negocio
class PostService:
    def __init__(self, db: Session):
        self.db = db

    def create(self, post_data: PostCreate) -> Post:
        """Crea un post con validaciones de negocio"""
        # Validar autor existe
        author = self.db.get(Author, post_data.author_id)
        if not author:
            raise ValueError("Author not found")

        # Validar autor activo
        if not author.is_active:
            raise PermissionError("Author is not active")

        # Procesar tags
        tags = self._get_or_create_tags(post_data.tag_names)

        # Crear post
        post = Post(
            title=post_data.title,
            content=post_data.content,
            author_id=post_data.author_id,
```

```

        tags=tags
    )
    self.db.add(post)
    self.db.commit()
    self.db.refresh(post)

    return post

```

Responsabilidades:

- Lógica de negocio
- Validaciones de dominio
- Orquestación de operaciones
- Manejo de transacciones

NO debe:

- Conocer detalles HTTP (códigos, headers)
- Importar FastAPI directamente
- Definir endpoints

7. Flujo de una Request

POST /posts {"title": "Hello", "author_id": 1}



```

Router: create_post()
1. Valida con PostCreate
2. Obtiene db session
3. Llama PostService.create()

```



```

Service: create()
1. Valida autor existe
2. Valida autor activo
3. Procesa tags
4. Crea Post
5. Commit

```



```

Router
Retorna PostResponse (201)

```

8. Manejo de Errores

El Service lanza excepciones Python, el Router las convierte a HTTP:

```

# services/post_service.py
class PostService:
    def create(self, post_data: PostCreate) -> Post:

```

```
author = self.db.get(Author, post_data.author_id)
if not author:
    raise ValueError("Author not found") # ← Excepción Python

# routers/posts.py
@router.post("/", response_model=PostResponse)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    service = PostService(db)
    try:
        return service.create(post)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e)) # ← HTTP Error
    except PermissionError as e:
        raise HTTPException(status_code=403, detail=str(e))
```

✓ Checklist

- ☐ Entiendo por qué separar capas
- ☐ Sé qué responsabilidad tiene cada capa
- ☐ Puedo identificar endpoints "gordos"
- ☐ Entiendo el flujo Router → Service → Model

[← Anterior: Queries](#) | [Siguiente: Implementando Servicios →](#)